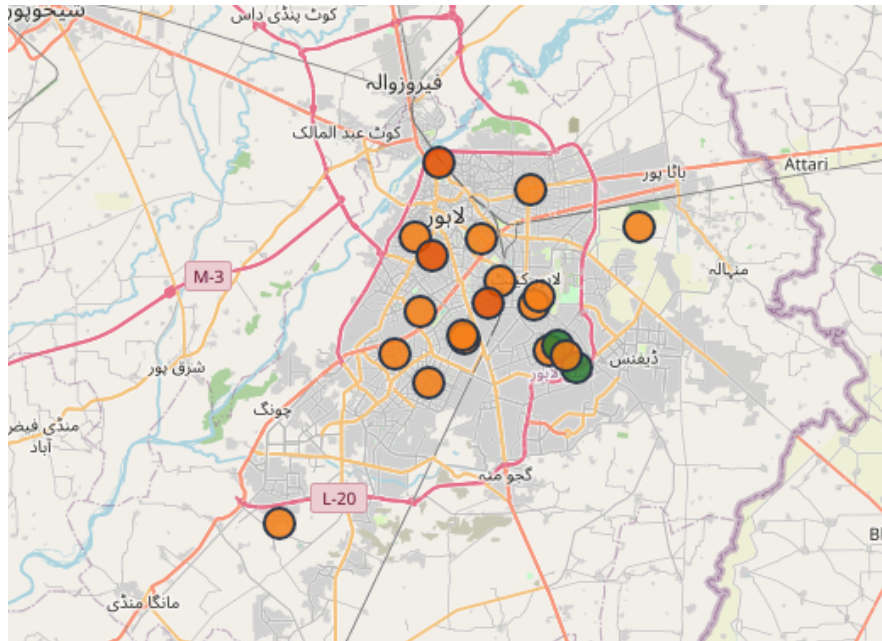


Lahore's City Wide 3 day AQI Forecaster based on realtime sensor data

Real time air quality forecasting across 19 locations in Lahore



Made by : Nawal Jabeen

Link: <https://pearlsaqipredictor-aqiforecasterbynawal.streamlit.app/>

Github Repo: https://github.com/Nawaljabeen/Pearls_AQI_Predictor

Note: Much work has gone into compiling this hand crafted document. Please credit if you plan to use the text.

Table of Contents

Lahore's City Wide 3 day AQI Forecaster based on realtime sensor data.....	1
Real time air quality forecasting across 19 locations in Lahore.....	1
Table of Contents.....	2
Executive Summary & Problem Statement.....	3
Data Research & Sourcing.....	4
Historical Backfilling.....	5
EDA & Feature Engineering.....	6
Model Training and Competitive Benchmarking.....	11
Sector Offset Layer(Localised AQI).....	12
Automated CI/CD Pipelines.....	13
Model Explainability.....	13
Interactive Dashboard & Deployment.....	15
Conclusion and Thoughts.....	17

Executive Summary & Problem Statement

With the massive increase in population and usage of resources, Air pollution has become a global environmental concern. According to the World Health Organization (WHO), air pollution levels are substantially increasing worldwide; more than 90% people breathe polluted air and approximately 7 million deaths are caused by outdoor and indoor aerosol pollutants on a yearly basis. Outdoor (ambient) air pollution due to high concentrations of different species, including airborne particulate matter (PM), ozone (O₃), nitrogen oxide (NO), nitrogen dioxide (NO₂), volatile organic compounds (VOC), carbon monoxide (CO), and sulfur dioxide (SO₂) have adverse health effects.

Lahore itself is known for being one of the most polluted cities. While EPA (environmental protection agency) of Punjab does have physical sensors installed across Lahore for real time particulate data, they are sparse, experience outages and lack developer friendly AQI's. Moreover, This project differentiates itself from conventional AQI forecasters like Open Meteo that derive their forecasts from massive numerical weather prediction and atmospheric composition models such as the Copernicus Atmosphere Monitoring Service (CAMS). Operating at coarse spatial resolutions of 11 to 40 kilometers per grid cell, these models simulate large-scale chemical transport and boundary-layer meteorology. However, because they average conditions over wide geographical areas, they often fail to capture hyper local anomalies. This is where the Pearls AQI Predictor Differentiates itself. By leveraging real time IoT sensor data from Clarity Movement Co, the Pearls AQI predictor predicts hyper local AQI across 19 locations of Lahore. Aqi varies widely across a city like Lahore, as a resident, i can personally confirm that air quality ranges across the many locations in Lahore. AQI in DHA may be vastly different from AQI in Johar Town. Hence it is essential to model AQI across Lahore and not just average it.

The Pearls AQI predictor is a serverless, cloud native MLOps framework. Github Actions is Utilized for hourly feature ingestion and daily model training. Hopsworx is utilised as a version control for feature groups and to register trained XGBoost models. Thirdly, using a monthly offset strategy for each of the 19 locations, the Pearls AQI predictor forecasts AQI across Lahore, but more on that later.

Data Research & Sourcing

Before starting this project i looked into major contributors of AQI. During my research i came across this table in an article talking about air quality assessment in Northern Pakistan.

Pollutants	Method	Instruments/Analyzers
Nitrogen Oxides	Reference Method RFNA-0809-186 by US EPA (40 CFR, Part 53)	NOx Analyzer, Ecotech, Australia
Sulfur Dioxide	Equivalent Method EQSA-0509-188 by US EPA (40 CFR, Part 53)	SO ₂ Analyzer, Ecotech, Australia
Carbon Monoxide	Reference Method RFCA-0509-174 by US EPA (40 CFR, Part 53)	CO Analyzer, Ecotech, Australia
Ozone	Equivalent Method EQOA-0809-187 by US EPA (40 CFR, Part 53)	Ozone Analyzer, Ecotech, Australia
Particulate Matter	Reference Method RFPS-0498-116 by US EPA (40 CFR Part 50)	PQ 200 BGI, USA
Black Carbon	Dual Spot Measurement Method	Aethalometer AE33, Magee Scientific, USA

This table helped to look into pollutants that contribute to the Air Quality. The second step was looking into the environmental influencers like temperature, humidity, wind speed etc.

Since I planned for this AQI Predictor project to be hyper localised from the getgo, it was essential that Data research and sourcing was carried out, because *garbage in, would be garbage out*. Initially I started off with Google AQI API, it appealed as it provides global coverage and with a spatial resolution of 500x500 meters and detailed pollutant breakdowns.

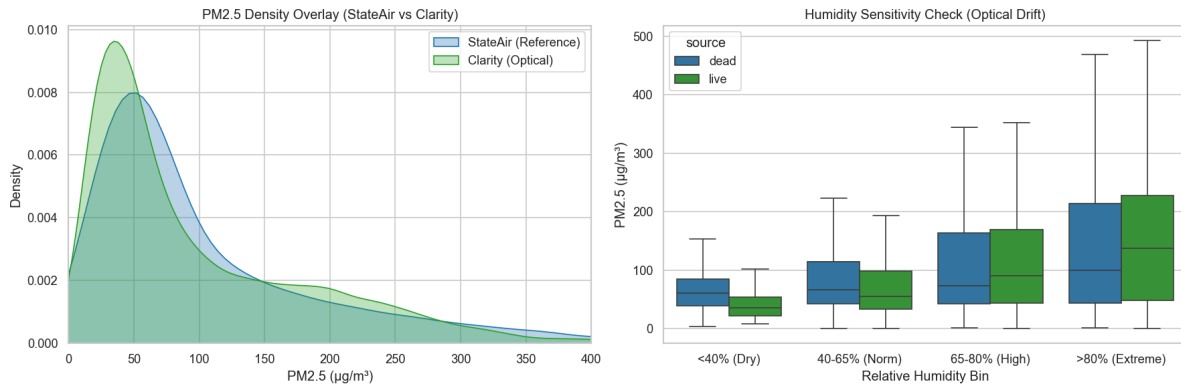
However, google AQI API was eventually dropped due to it only providing up to 30 days of historical data which was insufficient for training models.

Next I continued my research, stumbling upon EPA however their data was available in report format, in urdu and only up til Dec 2025 . This too was dropped. After further research about if Lahore has IoT sensors and partnerships I stumbled upon Open_AQ, which i found was an aggregator that pulled data from governmental AQI monitoring networks. I instantly got to work pulling a list of all their stations in Lahore. Then I ran a script to filter through finding stations with enough historical data. I found 2 stations:

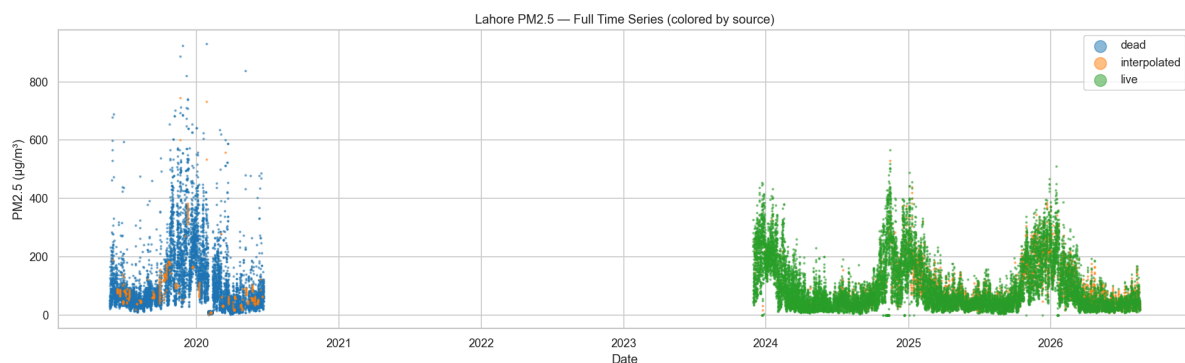
1. **Clarity**: Live station reporting since Nov 2023 and still live
2. **State Air**: Dead station with data from May 2019 to March 2025.

State Air was eventually dropped due to it being not being live anymore and merging it with Clarity caused major issues due to :

1. **A Two-Sample Kolmogorov-Smirnov** test was implemented to quantify the distribution similarity between the two datasets. The density overlay confirms this drift(as shown in figs below) , showing that Clarity and StateAir have fundamentally different density curves and peak concentrations. Training a predictive model on the old StateAir data would have caused it to misestimate current targets because the baseline statistical distributions do not match.



2. State Air sensors and Clarity react differently to environmental factors. They both report vastly different pm2.5 distributions , because of humidity driven hardware bias, As shown in the box plot.
3. Thirdly, State Air readings had a complete absence of pm2.5 data from 2023-2025 even though the Open_AQ list of stations reported it had data up til 2025, hence there was zero overlapping data to merge the 2.



Eventually I decided on using **Open Meteo for Weather** and **Open_AQ Clarity Stations data**.

Historical Backfilling

Once data sources were confirmed, i started off with ingestion scripts. Open AQ for pm2.5 readings and Open-meteo for hourly weather data. Weather data is pulled from both **Open meteo archive** and **openmeteo forecast api** (using its **past_days** parameter) because the archive api alone didnt have near real time data, it lagged behind by a few days. These 2 along with AQI data were merged.

The fetch layer pulled hourly pm2.5 readings , paginated into monthly chunks because early testing indicated that Open_AQ api was prone to timeout. Weather data on the

other hand was fetched per city from the Archive/Forecast split that I mentioned above.

Following this, a backfill process runs the entire fetch data+ feature pipeline , merges and writes the result to Hopsworks Feature Group. Since Clarity doesnt report every hour continuously short outages occur and gaps of upto 24 hours are interpolated, however if a row exceeds the 24h gap it is dropped. Every row holds a pm25_interpolated flag that indicates whether a value was a real value or if it was interpolated.

Onto the target and feature construction , **Lag Features** were computed for **1h, 3h, 6h, 24h, 48h, 168h** and **Rolling Statistics(3h and 24h mean and std)**. Moreover a **short term change rate, and forecast targets for 24h, 48h, 72h**.

Following is the backfill output:

```
! Lahore
Lahore station (Lahore (Clarity)): 18565 rows
Gap fill: 5580 hours interpolated/forward-filled, 0 hours still missing.

Merged: 24145 rows, 11 columns
After feature engineering: 23977 rows, 37 columns
city
Lahore    23977
Name: count, dtype: int64
Feature pipeline about to push, max timestamp: 2026-08-30 16:00:00+00:00
```

Starting at: 2023-11-01 -> 2026-08-30

Hours Interpolated : 5580

% of Dataset Interpolated : 23%

Rows After merge : 23977

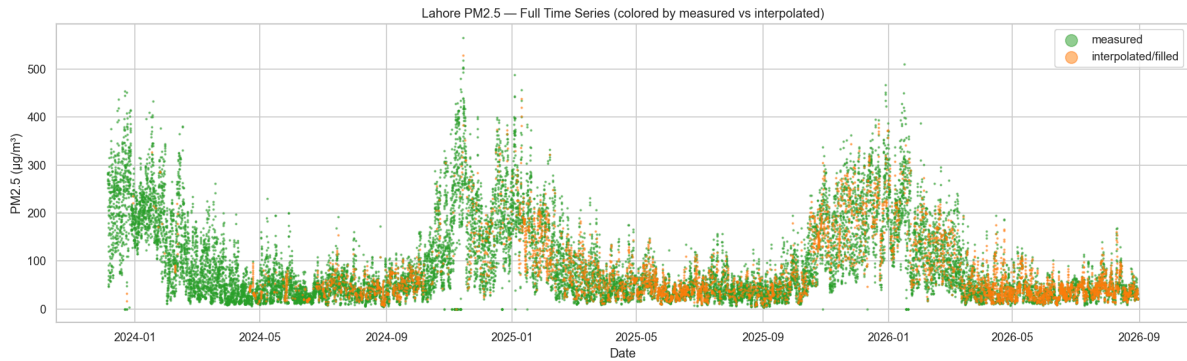
EDA & Feature Engineering

Exploratory data analysis was carried out of the full historical Clarity dataset to characterize seasonal, diurnal and cross source patterns before feature engineering, so that genuine data patterns would justify my feature choices.

1. Lahore pm2.5 full time series analysis

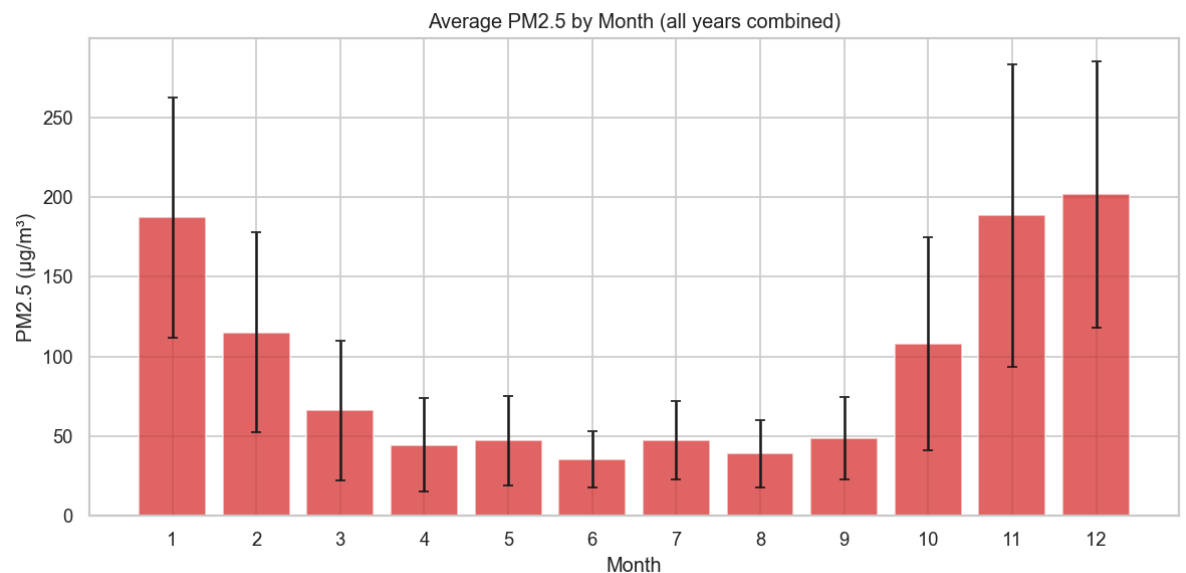
This time series visualization distinguished between real(green) and interpolated values(orange)

Highlights indicate seasonal volatility, with pm2.5 surging in smog and winter months whereas reducing in monsoon periods



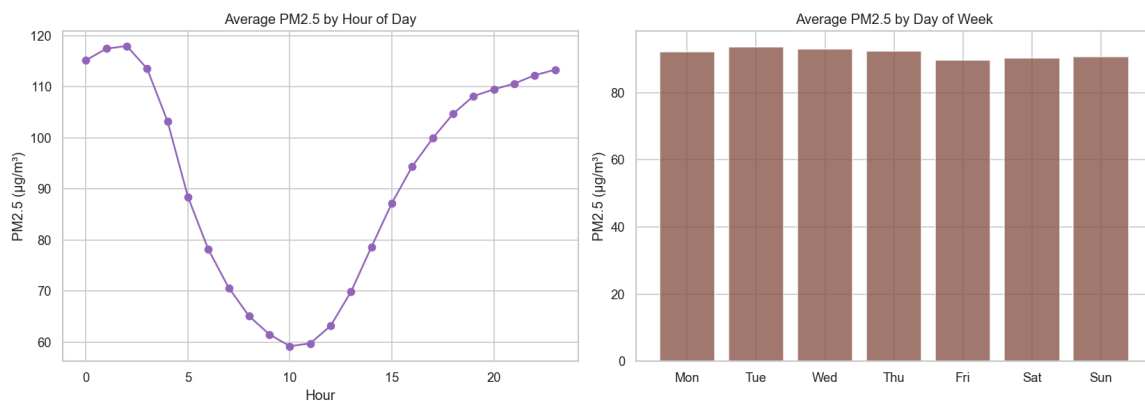
2. Average pm2.5 by month(years have been combined)

This plot shows average pm2.5 concentrations monthly along with standard deviation bars. This indicates seasonality, with pollution peaking in winter months(nov till january) as aforementioned for the time series analysis as well



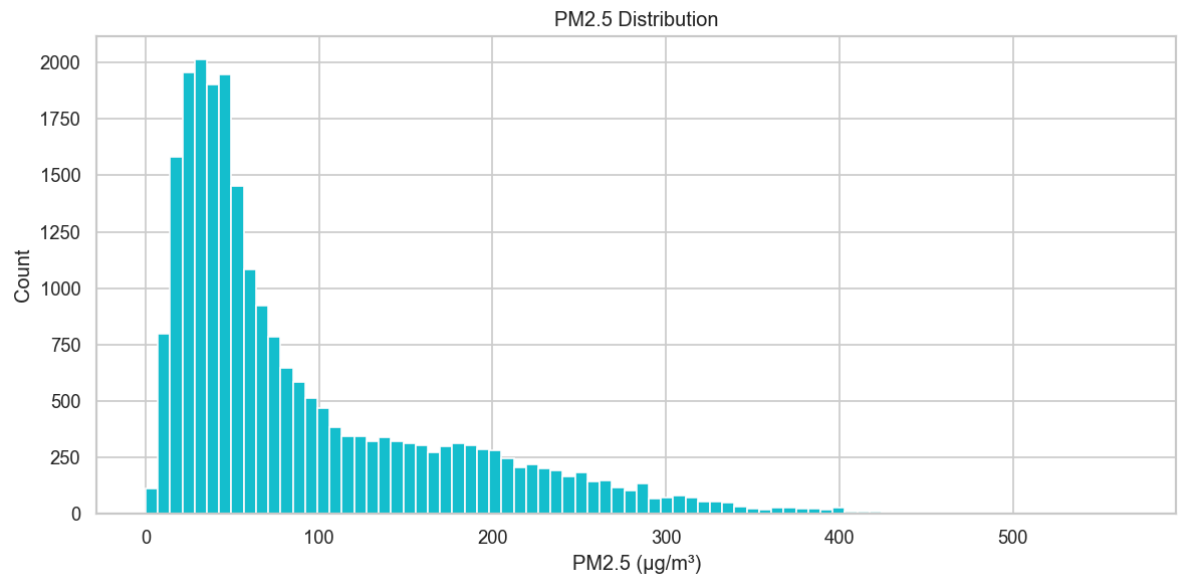
3. Average pm2.5 by Hour of day and Day of the week

Below are 2 subplots, one showing pm2.5 fluctuations on a line graph, varying by the hour, indicating that pollution is at peak in early morning hours and drop sharply to a daily min. Second shows by week, which remains relatively uniform.



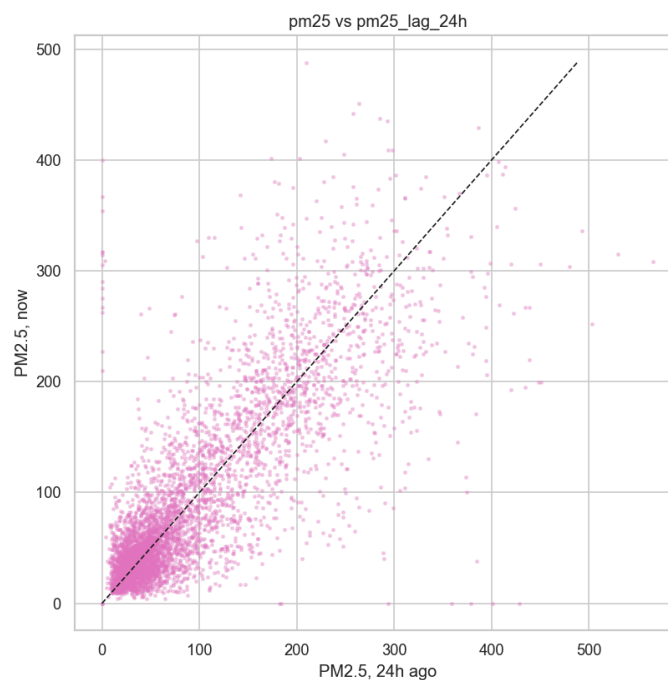
4. Pm2.5 Distribution

This next plot , a histogram shows the distribution and data density of recorded pm2.5 concentration values. The distribution is heavily right skewed where majority of recordings are below 100 and stretching to extreme pollution events above 400.



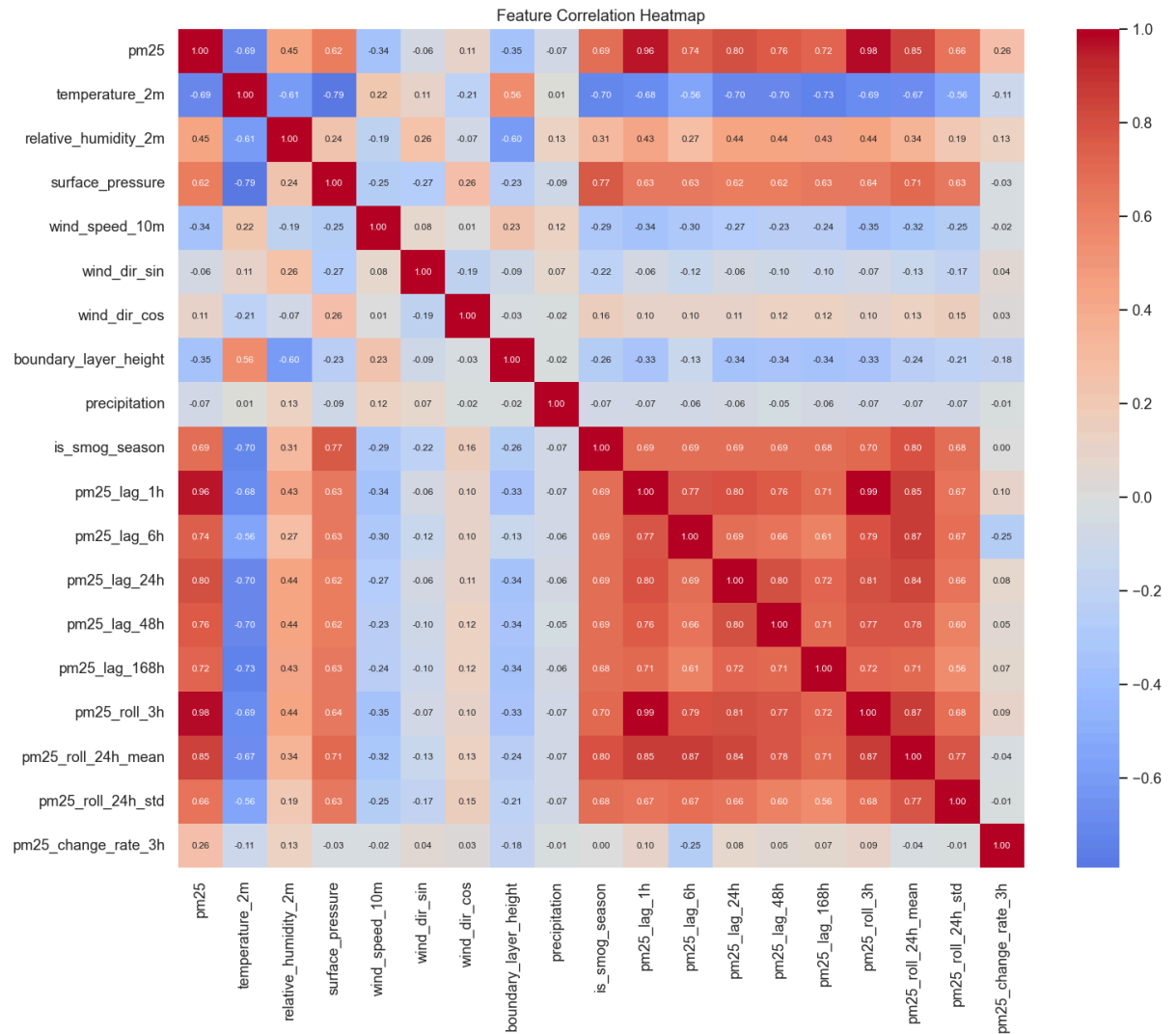
5. Pm2.5 vs pm2.5 lag_24h

The following is a scatter plot showing current pm2.5 values versus values from 24 hours ago. Illustrates strong persistence in atmospheric pollution levels from one day to the next, though significant dispersion away from the diagonal highlights rapid day-to-day meteorological shifts.



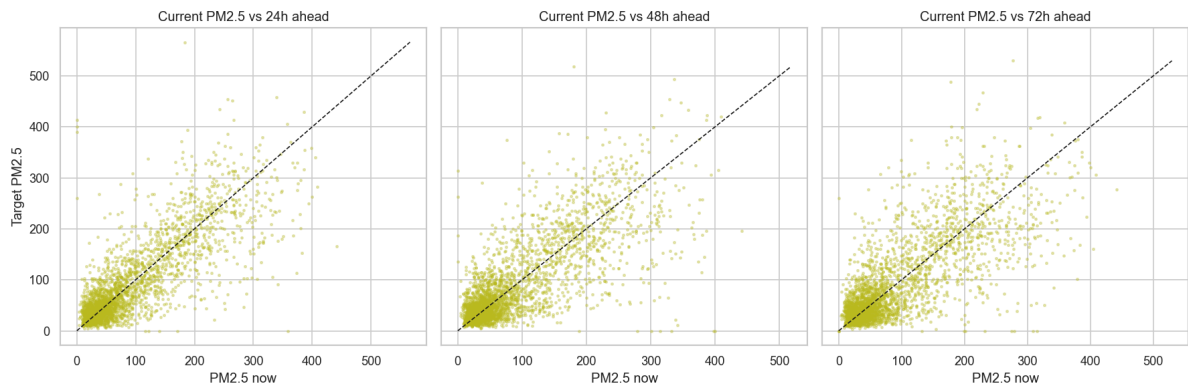
6. Feature Correlation Heatmap

The following is a detailed correlation matrix which is used to map linear relationships across weather parameters, time components, rolling stats and lag features. The heatmap confirms that lag features (1h and 3h) are positively correlated to pm2.5 while temperature show inverse correlation.



7. Current pm2.5 vs 24h, 48h and 72 ahead

The 3 scatterplots convey how pm2.5 measurements relate to future pm2.5 target concentrations. This shows that as the horizon increases so do the points scatter further from the reference line, which is normal and indicates that long range forecasting is uncertain.



All of these plots are available in the repository under the eda outputs folder.

Guided by the EDA findings, temporal features were encoded via sine/cosine pairs (**hour_sin**, **hour_cos**, **month_sin**, **month_cos**, **dow_sin**, **dow_cos**) because i wanted the model to treat for eg hour 23 and hour 0 as adjacent and not distant.

Furthermore, as mentioned b4r, **pm25_lag 24h**, **pm25_roll3h**, **pm25_change rate** captured recent history and trend direction.

Humidity interaction features like **rh_high_flag** and **pm25_rh_interaction** were added because of sensor cross validation finding.

Collinearity was handled, pm25 and pm25_roll_3h were seen as being highly correlated which is normal because one is a smoothed out version of the other. So , **pm25_deviation** was introduced as a derived feature : **pm25_deviation = pm25 - pm25_roll_3h**.

Model Training and Competitive Benchmarking

For Model Training , 3 Machine learning models were compared, **Ridge Regression**, **Random Forest**, and **XGBoost with K-Fold cross validation and hyperparameter optimization via Optuna and RandomizedSearchCV**. All 3 were evaluated and compared for **RMSE**, **MAE** and **R² score**.

Horizon	Model	RMSE	MAE	R2
24h	ridge	46.83	36.58	0.182
24h	random_forest	39.87	29.25	0.475
24h	xgboost	40.56	30.06	0.447
48h	ridge	50.43	39.80	0.045
48h	random_forest	43.22	32.26	0.409
48h	xgboost	43.79	32.67	0.398
72h	ridge	49.25	38.56	0.149
72h	random_forest	44.89	34.09	0.377
72h	xgboost	46.07	34.86	0.349

Both tree models are shown to substantially outperform ridge at all the horizons. It can be inferred that non linear relationships are expected in atmospheric data and the

ensemble methods performed better. **Random forest performed marginally better than XGBoost across all 3 horizons**

However, I selected XGBoost as the production model, because of 3 major reasons.

1. XGBoost handles both built in support for missing feature values whereas RF does not have this mechanism and requires complete inputs. Since my pipeline depends on real time sensor data which is prone to occasional gaps
2. Secondly, the SHAP based feature importance (refer **Explainability** section), was carried out on the XGboost model. Hence i retained the XGBoost model as the production model so that the results would be consistent
3. Thirdly, the margin between RF and XGBoost is small enough , both were legitimate choices but i choose XGBoost mostly because of reason 1.

Hyperparameter tuning was carried out for XGBoost using **Optuna** with a tree structured Parzen estimator sampler, running 15 trials per forecast horizon(24h, 48h, 72h). Every trial was then evaluated using a 5 fold time series split cross-validation.

Tuned search space covered :

- **N_estimators 100–600(step100)**
- **Max_depth 3–7**
- **Learning_rate 0.005–0.05 (log scale)**
- **Subsample 0.6–1.0**
- **Colsample_bytree 0.6–1.0**

The best performing parameter was then set per horizon (based on lowest mean CV RMSE) was used to train the final model. Trained models were then versioned and registered in the Hopsworks Model registry under **aqi_lahore_24h_xgboost, aqi_lahore_48h_xgboost, aqi_lahore_72_xgboost.**

Each registered model includes:

- The averaged cross-validated metrics (RMSE, MAE, R^2) as reported above
- An explicit input/output schema (via `hsml.schema.Schema / ModelSchema`), documenting the expected feature set and target
- A representative input example, used by Hopsworks for schema validation on downstream serving

Sector Offset Layer(Localised AQI)

After the base model was tuned, i had now 2 different approaches to introduce the localised AQI layer into my project. Initially i considered an independently modeled time series forecasting for each of the **19 AirGradient Stations** (also pulled from the list of OPEN_AQ stations CSV). Each of these Air Gradient stations have localised

data of upto **14 months and are still live**. The modelling approach was eventually discarded because 14 months of data was not enough to train the forecasting model. Furthermore, training 19 different models was going to cause massive overhead and even if they were all trained together it would cause pattern learning issues.

Instead i stuck to the simpler approach of **City baseline forecast(from clarity) + Learned local deviation** . Let me explain this simply. The offset model learns how much AQI deviates relative to the baseline (Clarity). For example it learns that DHA phase 5 is alot more cleaner than lets say, Ravi Road on windy days etc.

A simple additive function is used:

$$\text{Sector_Forecast} = \text{City_Base_Forecast} + \text{Sector_Offset}$$

Data is first ingested from the 19 AirGradient stations. It undergoes:

1. **Bounds checking:** any sensor reading below 0 or above 1000 is dropped because this would be an anomaly
2. **Un-duplication and Standardization:** All timestamps are converted to UTC and duplicated readings per sector are dropped.
3. **Storage:** The cleaned data is stores into the Hopsworks Feature store : **aqi_sector_features_fg**

Automated CI/CD Pipelines

To fullfil the serverless automated forecasting engine requirement of this project, **Github Actions** was used.

The CI/CD pipeline is divided into 3 schedule driven GitHub Actions workflows.

1. **Hourly Feature** ingestion that runs at start of ever hour. This runs the feature pipeline to fetch latest base AQI and weather readings , engineers the rolling window features and commits updated data to hospworks feature store.
2. **Hourly Batch Inference** that runs 30 mins after feature ingestion so that the feature store has been updated. It pulls latest models through the models registry and the generated forecasts are pushed to designated sink for frontend dashboard to use.
3. **Daily model training** to retrain the models on newly expanded dataset and pushes the new model back to hopsworks model registry.

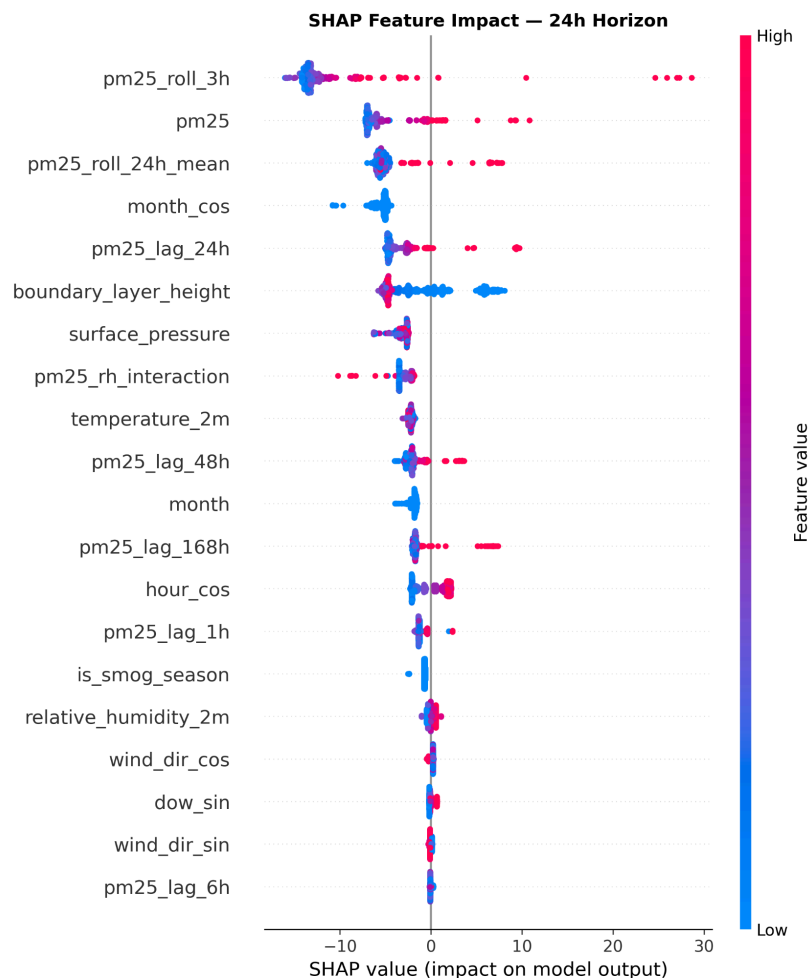
API keys were stored securely within the repository's Action secrets and locally in a .env file.

During the execution of YAML workflows these secrets are mapped directly to environment variables for eg, **env: HOPSWORKS_API_KEY: \${{ secrets.HOPSWORKS_API_KEY }}** . This keeps the codebase clean

Model Explainability

Beyond raw accuracy metrics, it is also important to verify if the model's predictions are driven by sensible relationships. 2 interpretability techniques were applied:

1. SHAP(Shapley Additive Explanations):
shap.TreeExplainer was applied to the most recent 500 hourly rows pulled from feature store.

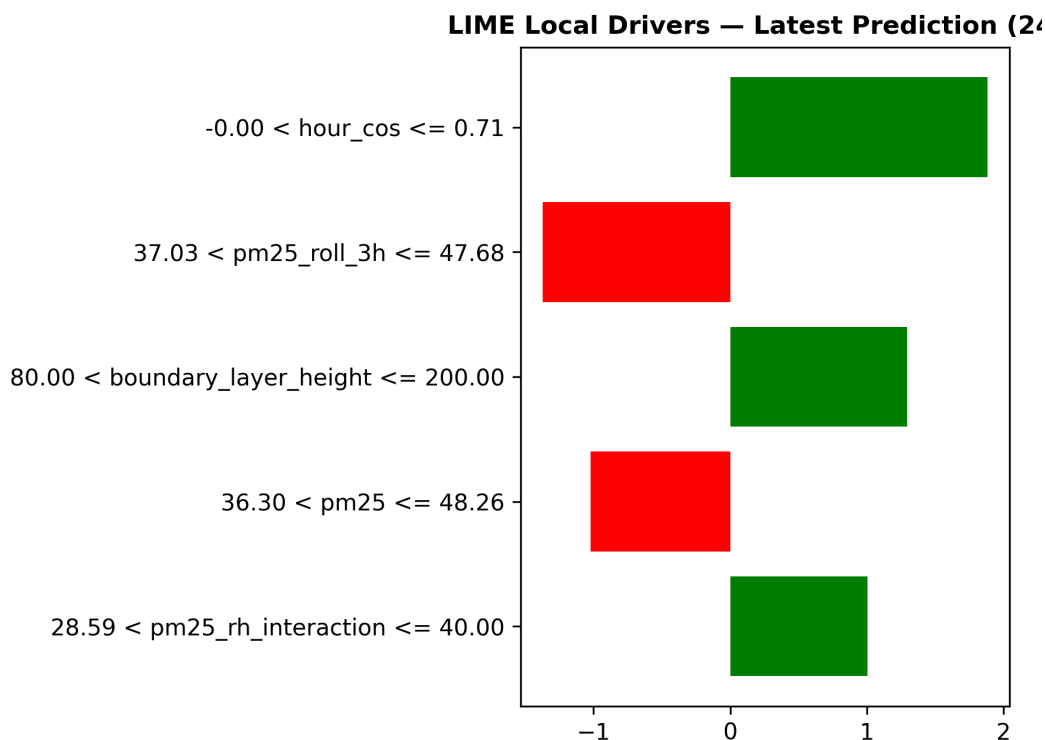


The **SHAP summary plot-24h** plots all 20 model features by their average impact on the predicted 24 pm2.5 value. Each point is one hour from the sample window, its horizontal position shows how much that feature pushed the prediction up or down for that hour, and its color shows whether the feature's own value was high (pink) or low (blue) at the time.

Dominant drivers : The top five features by a wide margin are all direct measures of recent PM2.5: **pm25_roll_3h (3-hour rolling average)**, the **instantaneous pm25**, **pm25_roll_24h_mean**, and **pm25_lag_24h**. This is expected for short-horizon air-quality forecasting — pollution is strongly autocorrelated, and what the air looks

like right now is the single best predictor of what it will look like tomorrow. Low recent readings (blue) consistently push the prediction down, elevated recent readings (pink) push it up, with the effect scaling noticeably for the more extreme (smog episode) values.

Boundary layer height : Low boundary layer height (blue) is associated with a *positive* push on predicted PM2.5, while high boundary layer height (pink) tends to pull predictions down. This matches the underlying atmospheric physics that a shallow boundary layer traps pollutants near the surface, while a deeper, well-mixed layer disperses them.



The **LIME local explanation plot** above, explains a single forecast. The local explanation shows :

- **Increasing the prediction:** `hour_cos` in a range corresponding to daytime hours, `boundary_layer_height` between 80–200 (on the lower end, consistent with the trapping effect identified in the SHAP analysis above), and `pm25_rh_interaction` in the 28.6–40 range.
- **Decreasing the prediction:** `pm25_roll_3h` in the 37–48 range and current `pm25` in the 36–48 range both moderate values for this location.

The presence of `boundary_layer_height` among the top local drivers, and in the same direction as the global SHAP pattern (low BLH → higher predicted PM2.5), is a

useful cross-check: the local explanation for this specific hour is consistent with the model's general behavior, not an outlier or artifact.

Interactive Dashboard & Deployment

The final stage of the pipeline pushes the model predictions to the end user through a **streamlit** web application ([app.py](#)). Where the earlier stages of the project (fetch → feature engineering → training → batch inference) are batch processes run on a schedule, this layer is the interactive, that turns those predictions into something a resident of Lahore can actually use to plan their day.

The dashboard reads from Hopsworks feature groups:

- **aqi_sector_predictions_fg** (v1): the latest 24h/48h/72h forecast for all 19 sectors, written directly by batch_inference.py.
- **aqi_base_lahore_fg** (v5): read for the single most recent row, used to construct a synthetic "Current (Live)" horizon that sits alongside the three forecast horizons.
- **aqi_sector_features_fg** (v1): historical sector-level readings, used to compute the same monthly mean offsets applied during batch inference, so the "Current (Live)" view is consistent with how the forecasted horizons are derived.

Data is fetched through the `fetch_predictions` function(). Radio buttons allow for selection between current , 24h, 48h and 72h views. Summary cards dynamically load according to location and horizon selected.

Perhaps the most interesting part , which I personally enjoyed the most was the **Interactive Folium Map** for the sector wise AQI. All 19 sectors plus the base AQI from clarity is plotted as color coded circle markers on the **OpenStreetMapBase** of Lahore. The circles use sector coordinates, By clicking or hovering on them the user can see the AQI and compare it to the **US EPA AQI severity scale**.

Selecting a sector updates the health guidance card accordingly , giving advice in plain user friendly language and visual icon so that non technical end users can interpret unassisted.

Under the map are cards showing cleanest versus worst AQI sector . At the very bottom sits a cloud synched inference table that provides the full prediction set into a by a horizon, by sector table.

The visual design is different from a typical data dashboard, It is meant to be soft, pastel and user friendly , soft shadows and a playful tone . This was something i wanted to do because air quality information should be actionable while being accurate and lowers the barrier between engaging with the data.

Conclusion and Thoughts

Overall, i really enjoyed doing this project . It allowed me the opportunity to build all the way from raw data to the interactive dashboard. I really wanted to add the by sector AQI, and I'm happy I didn't give up on it even after trying to train 19 models, one for each sector. Crazy, I know, but that's how I learnt.

In the start I was about to use google aqi api only to realize itd just give me 30 days of data , that too was a rollercoaster. I spent alot of my time on Data research but I'm glad i did.

At one point i messed up my trained models too, they were predicting the past lol. There were many issues faced but im glad i pushed through and i can say im happy with what ive made.